COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	07/10/2026	**DATE SUBMITTED:**	07/10/2026
NAME:	Macasaet, John David Milan, Kherwin Emmanuel Moradillo, Eunice	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Hands-on Activity 12.1 Algorithmic Strategies

1. PROCEDURE OUTPUT

What procedures did you perform?

STRATEGY	ALGORITHM	ANALYSIS
Recursion	Trees	By using the traversal method, the Depth First Search Traversal. Recursion is used where it explores as far as possible in each node along each branch before backtracking. This only stops when there is no unvisited node left.
Brute Force	Linear Search	Brute Force explores all possible solutions until the right one is found. Similarly to Linear Search, it iterates all the elements then checks the target element. If it's equal then it will return the current index of the element.
Backtracking	Trees	The Depth First Search Traversal or DFS uses backtracking to ensure every node is visited and goes back to the previous node and goes to the next branch.
Greedy	Selection Sort	In a greedy algorithm, it tries to build a solution one at a time and chooses a piece just like selection sort where it chooses a piece and places it permanently without the other elements.
Divide-and-Conquer	Sorting Algorithm	The Merge Sort is very similar to Divide-and-Conquer, they both divide the elements and half it into smaller portions until it's sorted or solved out, then recombined again.

Table 12-1. Algorithmic Strategies and Examples

SCREENSHOT	CODE:

	``cpp
	#include <iostream>
	#include <algorithm>
	
	int memo[1000001];
	
	int getMinSteps(int n)
	{
	    if(n == 1) return 0; //
	    if(memo[n] != -1) return memo[n]; //
	    int r = 1 + getMinSteps(n - 1); //
	    if(n%2 == 0) r = std::min(r, 1 + getMinSteps(n / 2));
	    if(n%3 == 0) r = std::min(r, 1 + getMinSteps(n / 3));
	    memo[n] = r; //
	    return r;
	}
	
	int main()
	{
	    int n;
	    std::cout<<"Insert n: ";
	    std::cin >> n;
	    for(int i = 0; i <= n; i++) memo[i] = -1;
	    std::cout << getMinSteps(n) << std::endl;
	    return 0;
	}
	``
	OUTPUT:
	``
	Insert n: 1
	0
	PS C:\Users\TI
	n } ; if ($?)
	Insert n: 4
	2
	PS C:\Users\TI
	n } ; if ($?)
	Insert n: 7
	3
	``

ANALYSIS	In this problem, memoization helps optimize the code. Instead of solving the same subproblem again and again, we use a memo, like a notebook, to store previous answers. When the same value comes up again, we just look it up in the memo, which makes the solution much faster.

Table 12-2. Memoization Implementation

SCREENSHOT	

	C/C++
	`#include <iostream>`
	`#include <algorithm>`
	
	`int getMinSteps(int n) {`
	`  int dp[n + 1], i;`
	`  dp[1] = 0; // trivial case`
	`  for (i = 2; i <= n; i++)`
	`  {`
	`  dp[i] = 1 + dp[i - 1];`
	`  if (i % 2 == 0) dp[i] = std::min(dp[i], 1 + dp[i / 2]);`
	`  if (i % 3 == 0) dp[i] = std::min(dp[i], 1 + dp[i / 3]);`
	`  }`
	`  return dp[n];`
	`}`
	
	`int main(){`
	`  int test[] = {1, 4, 7, 10};`
	`  for (int n : test)`
	`  std::cout << "n = " << n << ", minimum steps = " << getMinSteps(n) << std::endl;`
	`  return 0;`
	`}`
	
	`n = 1, minimum steps = 0`
	`n = 4, minimum steps = 2`
	`n = 7, minimum steps = 3`
	`n = 10, minimum steps = 3`

ANALYSIS	This one uses a loop and starts from 1 and goes up to n, so every smaller answer is already in the dp array when it is needed. There are no function calls, so there is no stack overflow problem

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PSEUDOCODE AND MANUAL COUNTING

```
START
FUNCTION findPathCount(grid, row, col, targetSum):
   // Base Case 1: Check if current indices are invalid or target sum is depleted below zero
   IF row < 0 OR col < 0 OR targetSum < 0 THEN
      RETURN 0
   END IF

   // Base Case 2: Arrived at origin cell (0, 0)
   IF row == 0 AND col == 0 THEN
      IF targetSum == grid[0][0] THEN
         RETURN 1
      ELSE
         RETURN 0
      END IF
   END IF

   // Recursive Step: Deduct current cell value and combine paths from above and left
   updatedSum = targetSum - grid[row][col]

   fromTop = findPathCount(grid, row - 1, col, updatedSum)
   fromLeft = findPathCount(grid, row, col - 1, updatedSum)

   RETURN fromTop + fromLeft
END FUNCTION
```

Manual Counting:
Path 1: (0,0) -> (0,1) -> (0,2) -> (1,2) -> (2,2) -> (3,2) -> (3,3)
4 + 7 + 1 + 3 + 1 + 6 + 3 = 25
Path 2: (0, 0) → (1, 0) → (1, 1) → (1, 2) → (2, 2) → (2, 3) → (3, 3)
4 + 7 + 1 + 3 + 1 + 6 + 3 = 25

CPP CODE

```cpp
#include <iostream>
#include <vector>
#include <algorithm>


// FUNCTION countPaths(matrix, m, n, cost):
int countPaths(const std::vector<std::vector<int>>& matrix, int m, int n, int cost) {

    /* Base Case 1: Out of matrix boundaries or negative remaining cost
     IF m < 0 OR n < 0 OR cost < 0 THEN
       RETURN 0 */


    if (m < 0 || n < 0 || cost < 0) {
        return 0;
    }

    /* Base Case 2: Reached starting cell (0,0)
    IF m == 0 AND n == 0 THEN
        IF cost == matrix[0][0] THEN
            RETURN 1
        ELSE
            RETURN 0 */

    if (m == 0 && n == 0) {
        if (cost == matrix[0][0]) {
            return 1;
        } else {
            return 0;
```

```
        } else {
            return 0;
        }
    }

    /* Recursive Step: Sum paths coming from Top (m-1, n) and Left (m, n-1)
    remainingCost = cost - matrix[m][n]
    */

    int remainingCost = cost - matrix[m][n];
    /* RETURN
        countPaths(matrix, m - 1, n, remainingCost) +
        countPaths(matrix, m, n - 1, remainingCost)
    END */
    return countPaths(matrix, m - 1, n, remainingCost) + countPaths(matrix, m, n - 1, remainingCost)


}

int main() {
    // Initialize the 4x4 Vector
    // Insert the given Values
    std::vector<std::vector<int>> matrix = {
        {4, 7, 1, 6},
        {6, 7, 3, 9},
        {3, 8, 1, 2},
        {7, 1, 7, 3}
```

```
    };

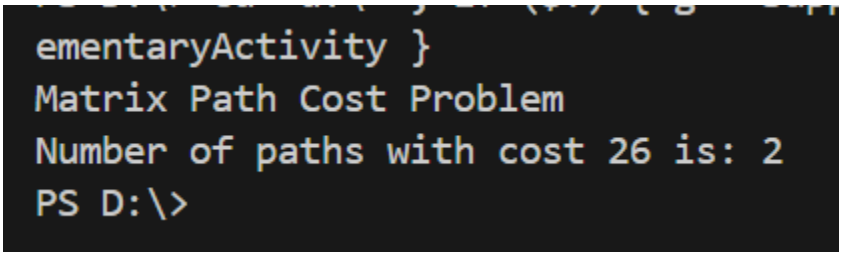
    // Get rows
    int rows = matrix.size();
    // Get Colums
    int columns = matrix[0].size();
    // Initialize the target Cost
    int targetCost = 26;

    // Initialize the Path to get the target cost
    int Paths = countPaths(matrix, rows - 1, columns - 1, targetCost);

    std::cout << "Matrix Path Cost Problem" << std::endl;
    std::cout << "Number of paths with cost " << targetCost << " is: " << Paths << std::endl;


    return 0;
}
```

SCREENSHOT OF OUTPUT



ANALYSIS	
MACASAET	The algorithm strategy used in the code is recursion. Recursion was used to move through the grid. The countPaths function is where recursion happens. To get all the valid paths, the code subtracts the current cell from the budget. Then it calls itself twice, once the cell above and the cell to the left. This way it branches out until there is nothing left to go to. At the end, if the program reaches (0,0) and the total is the budget, it will return a 1 meaning the budget was reached.
MILAN	The algorithm uses recursion to navigate through the grid. Inside the countPaths function, the program works backward from the bottom right corner to the starting cell (0, 0). At each step, it subtracts the current cell's value from the remaining budget. It then splits into two recursive calls, one moving to the cell above and one moving to the cell on the left, exploring all potential routes. This branching continues until every path hits a dead end (returning 0) or successfully reaches cell (0, 0) with the exact budget left (returning 1). Finally, the program adds up all the 1s returned by the successful branches to calculate the total number of valid paths.
MORADILLO	The code counts the ways to walk using recursion from the top left corner to the bottom right of the grid so that the number will step up to the target. The function calls itself, working backwards from the end and taking away each number and it stops when it goes outside the grid, runs out of number or reaches the start,

CONCLUSION	
MACASAET	In this activity, we learned about algorithm strategy and the many types in it. These algorithms strategies are ways to get a result, be it slower or in a faster way. Different strategies are used based on the situation or problem given. Mastering these strategies are a great way to better optimize your program.
MILAN	This activity highlighted the importance of algorithm strategies and their practical applications. These strategies offer distinct pathways to solve problems, balancing trade-offs between simple brute-force exploration and high-speed execution. Because every problem presents unique constraints, selecting the right strategy, such as recursion or dynamic

	programming is key. Overall, mastering these fundamental strategies is essential for building efficient, well-optimized, and scalable software.
MORADILLO	In this activity, I learned the different kinds of algorithm strategies and solved them in different ways too like recursion, brute force, backtracking, greedy, and divide and conquer. I also learned that memoization and bottom-up dynamic programming make code faster by saving answers, so the same work is not done again and again. In the supplemental activity, We used recursion to count the paths in a grid that add up to the target, and the program gave the correct answer. Recursion makes the problem easier to break down, but it repeats the same work, so it can become slow for big grids.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.